

РО 1.4. Основные сервисы Linux для предприятия

Лекция 1. Linux-сервер предприятия и первый web-сервис

Цель лекции

- Понять роль Linux-сервера в инфраструктуре предприятия.
- Объяснять сервисную модель Linux: пакет, конфигурация, служба, порт, журнал.
- Описывать базовый порядок публикации web-сервиса.
- Проверять службу, порт, firewall, web-доступ и журналы.

Список учебных вопросов

1. Linux-сервер в инфраструктуре предприятия.
2. Основные роли Linux-сервера: web, DNS, DHCP, LDAP, файловый сервер, router, VPN.
3. Сервисная модель Linux: пакет, конфигурация, служба, порт, журнал.
4. Подготовка серверной VM: hostname, статический IPv4-адрес, обновление пакетов.
5. Web-сервер как первый учебный сервис: Nginx или Apache.
6. Управление web-службой через systemctl.
7. Проверка доступности web-сервиса: ss, curl, браузер, firewall.
8. Журналы web-сервера и документирование параметров сервера.

1. Linux-сервер в инфраструктуре предприятия

Linux-сервер - это узел, который предоставляет сетевые или системные службы другим устройствам.

В предприятии сервер работает не сам по себе, а как часть инфраструктуры: клиенты, сеть, учетные записи, данные, безопасность.

Для администратора важно видеть сервер как управляемый объект: имя, адрес, роли, службы, журналы, обновления.

Первый web-сервис в этой лекции нужен как безопасный учебный пример публикации серверной роли.

Сервер как объект сопровождения

У сервера должно быть назначение: web, DNS, DHCP, файловый сервер или другая роль.

У сервера должны быть стабильные сетевые параметры, иначе клиенты не смогут надежно находить службу.

У сервера должна быть проверяемая конфигурация: что установлено, что запущено, какие порты открыты.

У сервера должны быть журналы, по которым можно восстановить события и найти ошибку.

Где Linux-сервер находится в сети

Клиент обращается к серверу по IP-адресу или имени.

Коммутатор обеспечивает связь внутри локальной сети.

Маршрутизатор или firewall связывает локальную сеть с другими сетями.

DNS помогает клиентам обращаться к серверу по имени.

Документация связывает технические параметры с назначением сервера.

Место Linux-сервера в инфраструктуре предприятия



Что значит подготовить сервер к роли

Серверная роль не начинается с установки пакета.

Сначала проверяют имя сервера, IP-адрес, доступность сети и обновление индекса пакетов.

Затем устанавливают нужную службу и проверяют ее состояние.

После этого открывают нужный порт и проверяют доступ с клиента.

В конце фиксируют результат в паспорте сервера.

2. Основные роли Linux-сервера

Один Linux-сервер может выполнять разные роли, но в учебной и реальной инфраструктуре роли нужно разделять логически.

Роль определяет, какие пакеты устанавливаются, какие порты открываются и какие журналы анализируются.

В этой РО роли будут изучаться постепенно: web, DHCP, DNS, LDAP, Samba, CA/TLS, rsyslog, router, VPN.

Понимание ролей помогает студенту видеть связь между отдельными лабораторными работами.

Web-роль

- Web-сервер публикует страницы и приложения по HTTP или HTTPS.
- HTTP - HyperText Transfer Protocol, протокол передачи web-страниц.
- HTTPS - HTTP поверх TLS, защищенный вариант web-доступа.
- Типичные пакеты: Nginx или Apache.
- Типичные порты: TCP 80 для HTTP и TCP 443 для HTTPS.

DNS и DHCP-роли

- DNS - Domain Name System, служба преобразования имен в IP-адреса.
- DHCP - Dynamic Host Configuration Protocol, служба автоматической выдачи сетевых параметров клиентам.
- DNS помогает обращаться к серверу по имени, а не по числовому адресу.
- DHCP выдает клиентам IP-адрес, gateway, DNS-сервер и доменное имя.
- В инфраструктуре эти роли часто связаны с адресным планом.

LDAP и файловая роль

- LDAP - Lightweight Directory Access Protocol, протокол доступа к каталогу учетных записей и групп.
- LDAP помогает централизовать пользователей и группы.
- Файловая роль предоставляет общий доступ к каталогам и данным.
- Samba публикует файловые ресурсы по SMB/CIFS для Linux- и Windows-клиентов.
- Для файловой роли особенно важны права доступа и резервное копирование.

Router, VPN и log-сервер

- Router или gateway передает трафик между сетями.
- VPN - Virtual Private Network, защищенный туннель для удаленного доступа.
- WireGuard будет использоваться как современный пример VPN.
- Log-сервер централизует журналы, чтобы администратор видел события с разных машин.
- Firewall ограничивает доступ к службам по правилам безопасности.



Основные роли Linux-сервера

Linux-сервер может выполнять разные роли в инфраструктуре предприятия.
Роль определяет установленные сервисы, открытые порты и задачи сервера.

Что такое роль сервера?

Роль сервера — это набор задач и сервисов, которые сервер выполняет для пользователей и других систем.

Каждая роль использует свои протоколы, порты, данные и журналы.



Зачем разделять роли?

- ✓ Проще администрирование и безопасность
- ✓ Легче масштабировать и обслуживать
- ✓ Меньше конфликтов между сервисами
- ✓ Проще резервное копирование и аудит
- ✓ Понятная архитектура инфраструктуры

Где применяются?

- Корпоративные сети
- Учебные лаборатории
- Хостинг и облачные платформы
- Филиалы и удалённые офисы

Популярные роли Linux-сервера

Web-сервер

Публикация сайтов и web-приложений.
Протоколы: HTTP, HTTPS
Пакеты: Nginx, Apache
Порты: 80 (HTTP), 443 (HTTPS)

Назначение: публикация контента и API

DNS-сервер

Разрешение имён в IP-адреса.
Протоколы: DNS (TCP/UDP)
Пакеты: BIND, PowerDNS
Порты: 53 (TCP/UDP)

Назначение: служба доменных имён

DHCP-сервер

Автоматическая выдача сетевых параметров.
Протоколы: DHCP (UDP)
Пакеты: isc-dhcp-server
Порты: 67 (UDP), 68 (UDP)

Назначение: выдача IP-адресов и настроек

LDAP-сервер

Централизованное хранение пользователей и групп.
Протоколы: LDAP, LDAPS
Пакеты: OpenLDAP
Порты: 389 (LDAP), 636 (LDAPS)

Назначение: каталог пользователей

Файловый сервер

Общий доступ к файлам и принтерам.
Протоколы: SMB/CIFS, NFS
Пакеты: Samba, NFS-server
Порты: 445 (SMB), 2049 (NFS)

Назначение: обмен файлами и ресурсами



Почтовый сервер

Отправка и приём электронной почты.
Протоколы: SMTP, IMAP, POP3
Пакеты: Postfix, Dovecot
Порты: 25, 587 (SMTP), 993 (IMAP), 995 (POP3)

Назначение: корпоративная почта

VPN-сервер

Безопасный удалённый доступ к сети.
Протоколы: OpenVPN, WireGuard, IPsec
Пакеты: openvpn, wg
Порты: зависит от протокола

Назначение: защищённый доступ

Маршрутизатор / Firewall

Маршрутизация трафика и фильтрация.
Протоколы: IP, ICMP, NAT
Пакеты: iptables/nftables, iproute2
Порты: любые (по правилам)

Назначение: управление трафиком и безопасностью

Журналирование и мониторинг

Сбор и анализ логов, мониторинг состояния.
Пакеты: rsyslog, syslog-ng, Prometheus, Grafana
Порты: 514 (syslog), 9090 (Prometheus)

Назначение: контроль и диагностика

Резервное копирование

Хранение и управление резервными копиями.
Пакеты: rsnapshot, borgbackup, restic
Порты: зависит от решения

Назначение: защита данных

Общие принципы работы с ролями



1. Определить роль

Понять, какие задачи будет выполнять сервер.



2. Установить пакеты

Установить нужные сервисы и утилиты.



3. Настроить сервисы

Изменить конфигурацию под требования.



4. Открыть порты

Разрешить только необходимый трафик.



5. Проверить работу

Проверить доступность и журналов.



6. Документировать

Зафиксировать параметры и назначение сервера.

Наиболее используемые порты

Порт	Протокол	Назначение
22	TCP	SSH (удалённое управление)
53	TCP/UDP	DNS
80	TCP	HTTP
443	TCP	HTTPS
445	TCP	SMB (файловый доступ)
389	TCP/UDP	LDAP
25, 587	TCP	SMTP (почта)
514	UDP	Syslog (журналы)
2049	TCP/UDP	NFS (файловый доступ)

Важно помнить



Открывайте только те порты, которые действительно нужны.



Регулярно обновляйте систему и устанавливайте обновления.



Проверяйте журналы — они помогут найти проблему.



Делайте резервные копии важных данных и конфигураций.

Почему не стоит смешивать все роли без плана

- Каждая роль открывает свои порты и создает свои риски.
- Сбой одной службы может повлиять на другие службы на том же сервере.
- Диагностика становится сложнее, если неизвестно, какие роли установлены.
- В учебной среде роли можно совмещать, но их нужно документировать отдельно.

3. Сервисная модель Linux

Сервисная модель Linux показывает, из каких частей состоит серверная роль.

Пакет устанавливает программу и служебные файлы.

Конфигурация определяет поведение службы.

Служба запускается и управляется через `systemd`.

Порт принимает сетевые подключения, а журнал фиксирует события и ошибки.

Пакет и конфигурация

Пакет - управляемая единица установки программы в Linux.

Для Debian/Ubuntu используется пакетный менеджер APT.

Конфигурационные файлы обычно находятся в **/etc**.

Для web-сервера конфигурация определяет, какой каталог публиковать и какие порты слушать.

Нельзя менять конфигурацию вслепую: после правки нужна проверка и перезапуск или reload службы.

Служба и systemd

Служба - фоновая программа, которая предоставляет функцию системе или клиентам.

systemd - система инициализации и управления службами в современных Linux-дистрибутивах.

Команда systemctl используется для проверки, запуска, остановки и включения служб в автозагрузку.

Для web-сервера важны состояния active, inactive и failed.

Порт и сетевой доступ

Порт - числовой идентификатор сетевой службы на сервере.

TCP - Transmission Control Protocol, надежный транспортный протокол для web-доступа.

Web-сервер обычно слушает TCP 80 для HTTP или 443 для HTTPS.

Если служба запущена, но порт закрыт firewall, клиент все равно не подключится.

Поэтому проверяют и службу, и порт, и сетевой доступ с клиента.

Журнал как источник диагностики

Журнал хранит события службы: успешные обращения, ошибки, предупреждения.

Для web-сервера важны access log и error log.

Access log показывает обращения клиентов.

Error log помогает найти ошибки конфигурации, прав доступа или файлов.

Администратор не должен ограничиваться фразой 'не работает': нужно найти запись, которая объясняет сбой.



СЕРВИСНАЯ МОДЕЛЬ LINUX

Сервис в Linux — это связка: пакет → конфигурация → служба → порт → журнал.



Понимая эту модель, администратор может установить, настроить, запустить, проверить и сопровождать любой сервис.

1 ПАКЕТ (Package)

Программное обеспечение устанавливается из репозитория.



Примеры пакетов:

- nginx
- apache2
- bind9
- openssh-server

```
# Установка пакета  
sudo apt install nginx
```

2 КОНФИГУРАЦИЯ (Configuration)

Файлы конфигурации определяют, как сервис должен работать.



Примеры путей:

- /etc/nginx/nginx.conf
- /etc/apache2/apache2.conf
- /etc/bind/named.conf
- /etc/ssh/sshd_config

```
# Проверка синтаксиса конфигурации  
sudo nginx -t
```

3 СЛУЖБА (Service)

Служба (демон) — это запущенный процесс, который предоставляет сервис.



Управление службой через systemd:

- `sudo systemctl start nginx`
- `sudo systemctl stop nginx`
- `sudo systemctl restart nginx`
- `sudo systemctl enable nginx`
- `sudo systemctl status nginx`

4 ПОРТ (Port)

Сервис слушает сетевой порт и принимает соединения.



Примеры сервисов и портов:

- HTTP → TCP 80
- HTTPS → TCP 443
- SSH → TCP 22
- DNS → TCP/UDP 53

```
# Проверка портов  
ss -tulnp | grep LISTEN
```

5 ЖУРНАЛ (Journal / Log)

Журналы фиксируют события, ошибки и полезную информацию.



Где смотреть логи:

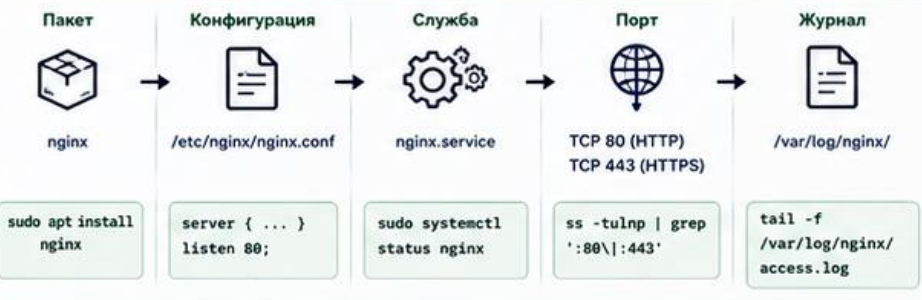
- `journalctl -u nginx`
- `/var/log/nginx/access.log`
- `/var/log/nginx/error.log`
- `journalctl -xe`

```
# Просмотр последних записей  
sudo journalctl -u nginx -n 50
```

КАК ВСЁ РАБОТАЕТ ВМЕСТЕ



ПРИМЕР: WEB-СЕРВЕР NGINX



ПРОВЕРКА СЕРВИСА: БЫСТРЫЙ ЧЕК-ЛИСТ

1	Пакет установлен?	<code>dpkg -l grep nginx</code>
2	Конфигурация корректна?	<code>nginx -t</code>
3	Служба запущена?	<code>systemctl status nginx</code>
4	Порт открыт и слушается?	<code>ss -tulnp grep :80</code>
5	Сервис отвечает клиенту?	<code>curl -I http://localhost</code>

РЕЗУЛЬТАТ



Сервис работает корректно



Найдена проблема —
смотрите журналы
и конфигурацию

ВАЖНЫЕ ПРИНЦИПЫ



Безопасность
Открывайте только необходимые порты.
Ограничивайте доступ.



Прозрачность
Документируйте конфигурацию, порты и зависимости.



Наблюдаемость
Используйте журналы и мониторинг для выявления проблем.



Управляемость
Используйте `systemd` и конфигурационные файлы,
а не ручные правки в процессе.



СОВЕТЫ



Изменили конфигурацию — проверяйте синтаксис и перезапускайте службу.



Следите за журналами сразу, это экономит время.



Используйте `systemctl enable` для автозапуска нужных сервисов.



Делайте резервные копии конфигураций перед изменениями.

4. Подготовка серверной VM

Серверная VM должна быть предсказуемой: понятное имя, постоянный адрес, актуальный индекс пакетов.

VM - Virtual Machine, виртуальная машина, которая имитирует отдельный компьютер.

Перед установкой web-службы администратор проверяет базовую готовность сервера.

Такой порядок снижает количество случайных ошибок в работе.

Настройка и проверка hostname

Hostname - системное имя узла, которое отображается в shell, журналах и документации.

Для просмотра имени используют: `hostnamectl`.

Ожидаемый результат: в поле Static hostname видно осмысленное имя, например `web01`.

Для изменения имени используют: **`sudo hostnamectl set-hostname web01`**.

После изменения нужно снова проверить `hostnamectl` и зафиксировать имя в паспорте сервера.

Статический IPv4-адрес

IPv4 - Internet Protocol version 4, адресация вида 192.168.10.20.

Для сервера предпочтителен постоянный адрес, потому что клиенты и документация должны ссылаться на стабильный узел.

Для просмотра адресов используют: `ip addr`.

Ожидаемый результат: на рабочем интерфейсе есть `inet`-адрес с `prefix`, например 192.168.10.20/24.

Если адрес получен по DHCP, нужно понимать, закреплен ли он `reservation` или задан вручную.

Проверка маршрута и DNS перед установкой

Перед установкой пакетов сервер должен иметь маршрут к репозиториям и рабочий DNS.

Для маршрута используют: `ip route`.

Ожидаемый результат: есть строка `default via адрес_gateway`.

Для проверки DNS можно использовать: `resolvectl query deb.debian.org`.

Если имена не разрешаются, `apt install` может завершиться ошибкой загрузки пакетов.

Обновление индекса пакетов

APT - Advanced Package Tool, пакетный менеджер Debian/Ubuntu.

Перед установкой нового пакета обновляют локальный индекс: `sudo apt update`.

Эта команда не обновляет программы, а скачивает свежий список доступных пакетов.

В выводе не должно быть ошибок Err, связанных с сетью, DNS или подписью репозитория.

Если apt update прошел с ошибками, установку web-сервера сначала откладывают и исправляют причину.



ПОДГОТОВКА LINUX-СЕРВЕРА

перед установкой службы

Правильно подготовленный сервер — это стабильность, безопасность и предсказуемость работы любой службы.



Безопасность



Стабильность



Управляемость

ЗАЧЕМ ГОТОВИТЬ СЕРВЕР?

- ✓ Единые стандарты именования и адресации.
- ✓ Актуальные обновления безопасности.
- ✓ Корректные время и DNS — основа работы служб.
- ✓ Минимум лишнего софта — меньше рисков.
- ✓ Чистая среда — проще сопровождение и отладка.

МИНИМАЛЬНЫЕ ТРЕБОВАНИЯ

- Рекомендуемые ресурсы (для учебных целей)
- CPU: 2 vCPU и выше
 - RAM: 2-4 GB
 - Диск: 20 GB+
 - Доступ: консоль/SSH с правами sudo

ПОРЯДОК ПОДГОТОВКИ СЕРВЕРА

1 ПРОВЕРКА ПОДКЛЮЧЕНИЯ

Убедитесь, что сервер доступен по сети.



```
$ ping -c 3 <IP_сервера>
$ ssh user@<IP_сервера>
```

2 УСТАНОВКА ИМЕНИ ХОСТА

Имя сервера должно быть понятным и уникальным.

host01.example.local

```
# временно
$ sudo hostnamectl set-hostname host01

# постоянно
$ sudo vi /etc/hostname
host01.example.local

$ sudo vi /etc/hosts
127.0.0.1 localhost
127.0.1.1 host01.example.local
```

3 НАСТРОЙКА СЕТЕВОГО АДРЕСА

Настройте статический IPv4-адрес, шлюз и DNS.



Пример для netplan (Ubuntu):

```
$ sudo vi /etc/netplan/01-netcfg.yaml

network:
  version: 2
  ethernet:
    ens160:
      dhcp4: no
      addresses: [192.168.10.20/24]
      gateway4: 192.168.10.1
      nameservers:
        addresses: [192.168.10.2, 8.8.8.8]
```

```
$ sudo netplan apply
```

4 ОБНОВЛЕНИЕ ПАКЕТОВ

Обновите индекс репозитория и установленные пакеты.



```
# Debian / Ubuntu
$ sudo apt update
$ sudo apt upgrade -y

# RHEL / CentOS / Rocky
$ sudo dnf update -y
```

5 УСТАНОВКА БАЗОВЫХ УТИЛИТ

Установите утилиты, которые понадобятся для диагностики и администрирования.



```
# Debian / Ubuntu
$ sudo apt install -y \
vim curl wget net-tools \
dnsutils iproute2 \
lsuf htop unzip ufw

# RHEL / CentOS / Rocky
$ sudo dnf install -y \
vim curl wget net-tools \
bind-utils iproute \
lsuf htop ufw
```

6 ПРОВЕРКА КЛЮЧЕВЫХ ПАРАМЕТРОВ

- Имя хоста
`hostnamectl`
- IP-адрес
`ip -4 addr`
- Маршрут по умолчанию
`ip route`
- DNS-серверы
`resolvectl status`
- Время и синхронизация
`timedatectl status`
- Открытые порты
`ss -tuln`

```
$ hostnamectl
Static hostname: host01.example.local
P ip -4 addr show ens160
inet 192.168.10.20/24 brd 192.168.10.255 ...
$ ip route
default via 192.168.10.1 dev ens160
$ resolvectl status
DNS Servers: 192.168.10.2
8.8.8.8
$ timedatectl status
NTP synchronized: yes
$ ss -tuln
State Recv-Q Send-Q Local Address:Port
LISTEN 0 128 0.0.0.0:22
```

7 БАЗОВАЯ БЕЗОПАСНОСТЬ

- Создайте пользователя для администрирования
- Используйте ключи SSH (аутентификация по ключу)
- Отключите вход root по паролю
- Настройте UFW (или firewall)
- Проверьте статус службы SSH

```
# создать пользователя
$ sudo adduser adminuser
$ sudo usermod -aG sudo adminuser

# ключи SSH
$ ssh-keygen -t ed25519
$ ssh-copy-id adminuser@host01

# запрет входа root по паролю
$ sudo vi /etc/ssh/sshd_config
PermitRootLogin no
PasswordAuthentication no
$ sudo systemctl reload sshd

# firewall (пример)
$ sudo ufw allow OpenSSH
$ sudo ufw enable
$ sudo ufw status
```

8 ВРЕМЯ И DNS — ОСНОВА ВСЕГО

- Время
- Корректное время важно для сертификатов, логов и аутентификации.

```
$ timedatectl set-ntp true
$ timedatectl status
```

- DNS
- Проверьте разрешение имен как для внешних, так и для внутренних ресурсов.

```
$ ping -c 2 google.com
$ nslookup example.com
$ getent hosts example.com
```

9 ЧИСТАЯ СРЕДА

- Удалите ненужные пакеты и службы.
- Отключите неиспользуемые службы.
- Освободите место на диске.

```
$ sudo systemctl list-unit-files --state=enabled
$ sudo systemctl disable <service>
$ sudo apt autoremove -y
$ df -h
```

10 ФИНАЛЬНАЯ ПРОВЕРКА

- ✓ Сервер доступен по SSH.
- ✓ Имя хоста задано и разрешается.
- ✓ Статический IP-адрес работает.
- ✓ Шлюз и DNS настроены корректно.
- ✓ Пакеты обновлены.
- ✓ Время синхронизировано.
- ✓ Базовые утилиты установлены.
- ✓ Firewall активен.
- ✓ Журналы не содержат ошибок.

ДОКУМЕНТАЦИЯ СЕРВЕРА

Зафиксируйте параметры в паспорте сервера:

- Имя хоста:
- IP-адрес / маска:
- Шлюз:
- DNS:
- Назначение сервера:
- Ответственный:
- Дата подготовки:

ПОЛЕЗНЫЕ СОВЕТЫ

- Используйте осмысленные имена: web01, dns01 и т.д.
- Всегда работайте под обычным пользователем через sudo.
- Регулярно обновляйте систему безопасности.
- Храните конфигурации под контролем (git, backup).



ВАЖНО ПОМНИТЬ

Подготовка сервера — обязательный шаг перед установкой любой службы. Экономит время, предотвращает ошибки и упрощает сопровождение.

ИТОГОВАЯ ЦЕПОЧКА



Доступность



Имя и сеть



Обновления



Утилиты



Безопасность



Проверки



Документация



Готов к службе

Типовые ошибки подготовки

- Сервер оставлен с именем localhost или ubuntu, и в отчетах непонятно, о какой машине идет речь.
- IP-адрес изменяется после перезагрузки, потому что не закреплен в настройках или DHCP reservation.
- DNS не работает, но студент сразу пытается установить пакет.
- apt update завершился ошибками, но установка продолжается без анализа вывода.
- Паспорт сервера не ведется, поэтому результат невозможно быстро проверить повторно.

5. Web-сервер как первый учебный сервис

Web-сервер удобен как первая серверная роль: его легко установить, проверить и показать клиенту.

Он демонстрирует полный цикл: пакет, конфигурация, служба, порт, firewall, журнал.

В учебной работе можно использовать Nginx или Apache.

Главная цель сейчас - не сложная web-разработка, а публикация и проверка сетевой службы.

Nginx и Apache: базовое сравнение

Nginx часто используют как быстрый web-сервер и reverse proxy.

Apache HTTP Server - зрелый web-сервер с развитой модульной системой.

Оба могут публиковать статическую HTML-страницу.
В Debian/Ubuntu пакеты обычно называются **nginx** и **apache2**.

Для первой лабораторной достаточно выбрать один web-сервер.

Установка web-сервера

Перед установкой уже должен быть выполнен `sudo apt update`.

Для установки Nginx используют: `sudo apt install nginx`.

Для установки Apache используют: `sudo apt install apache2`.

APT покажет список пакетов и объем загрузки; перед подтверждением нужно прочесть изменения.

После установки переходят не к браузеру, а сначала к проверке службы.

Где лежит первая web-страница

Document root - каталог, из которого web-сервер отдает файлы клиенту.

Для Nginx в Debian/Ubuntu часто используется /var/www/html.

Для Apache в Debian/Ubuntu также часто используется /var/www/html.

Простой файл index.html позволяет проверить публикацию без разработки приложения.

Если страница не меняется в браузере, нужно проверить путь, кэш браузера и права на файл.

Создание простой HTML-страницы

Для учебной проверки можно создать файл `/var/www/html/index.html`.

Пример безопасной ручной правки: `sudo nano /var/www/html/index.html`.

Внутри достаточно указать заголовок сервера и номер лабораторной работы.

После сохранения файл проверяют локально через `curl`.

Если web-сервер возвращает старую страницу, нужно убедиться, что изменен правильный `document root`.



ПУТЬ HTTP-ЗАПРОСА К WEB-СТРАНИЦЕ

Что происходит от ввода адреса в браузере до отображения страницы



Клиентская
сторона



Сетевая
инфраструктура



Linux-сервер (Web)

1 Ввод адреса в браузере

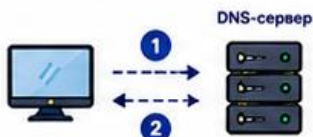
Пользователь вводит URL или нажимает ссылку.



Браузер определяет протокол (HTTP/HTTPS), хост и порт (по умолчанию 80 или 443).

2 Проверка кэша и DNS

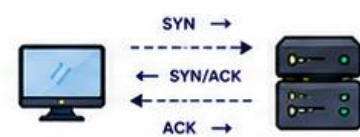
Браузер проверяет кэш DNS. Если записи нет — выполняется DNS-запрос к DNS-серверу.



Пример:
www.example.com → 192.168.10.20

3 Установка соединения

Браузер устанавливает TCP-соединение с IP-адресом сервера и нужным портом.



Порты:
• HTTP → TCP 80
• HTTPS → TCP 443 (после TLS)

4 Отправка HTTP-запроса

Браузер отправляет HTTP-запрос (или HTTPS-запрос после TLS-рукопожатия).

```
GET /index.html HTTP/1.1
Host: www.example.com
User-Agent: Mozilla/5.0
Accept: text/html
Connection: keep-alive
}
```

Запрос содержит метод (GET), путь, заголовки и служебные данные.

5 Обработка на web-сервере

Web-сервер (Nginx/Apache) принимает запрос и ищет нужный ресурс.



6 Отправка HTTP-ответа

Сервер возвращает HTTP-ответ с кодом состояния и содержимым.

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8
Content-Length: 1024

<!DOCTYPE html>
<html>...
</html>
```

Код 200 OK — всё успешно.
В ответе — заголовки и данные.

7 Отображение страницы

Браузер получает ответ, разбирает HTML, загружает дополнительные ресурсы (CSS, JS, изображения) и отображает страницу пользователю.



Для каждого дополнительного ресурса могут быть новые запросы (шаги 2–6 повторяются).

СХЕМА ВЗАИМОДЕЙСТВИЯ



ЧТО ПРОИСХОДИТ ВНУТРИ СЕРВЕРА

- Приём запроса
Nginx/Apache принимает соединение и читает HTTP-запрос.
- Маршрутизация
Определяется виртуальный хост, корневая директория и нужный ресурс.
- Генерация ответа
Файл отдаётся как есть или выполняется скрипт/приложение.
- Формирование HTTP-ответа
Добавляются заголовки, код состояния и содержимое.

ПОЛЕЗНЫЕ КОМАНДЫ ДЛЯ ПРОВЕРКИ

Проверка DNS:
\$ dig www.example.com
\$ nslookup www.example.com

Проверка соединения и портов:
\$ ping www.example.com
\$ ss -tuln | grep -E':80|:443'

Проверка HTTP-ответа:
\$ curl -I http://www.example.com
\$ curl -I https://www.example.com

Просмотр логов сервера:
\$ sudo tail -f /var/log/nginx/access.log
\$ sudo tail -f /var/log/nginx/error.log

КОДЫ СОСТОЯНИЯ HTTP (часто используемые)

200 OK	Запрос успешно выполнен
301 Moved Permanently	Ресурс перенесён навсегда
404 Not Found	Ресурс не найден
403 Forbidden	Доступ запрещён
500 Internal Server Error	Внутренняя ошибка сервера

ТИПИЧНЫЕ ПОРТЫ

80/tcp	HTTP
443/tcp	HTTPS (HTTP over TLS)
22/tcp	SSH (управление сервером)
53/udp,tcp	DNS
123/udp	NTP

ВАЖНО ПОМНИТЬ

- DNS должен корректно резолвить имя в IP-адрес.
- Порт на сервере должен быть открыт.
- Firewall не должен блокировать соединение.
- Web-сервис и конфигурация должны работать.
- Логи — ваш главный инструмент диагностики.

6. Управление web-службой через `systemctl`

Установка пакета не гарантирует, что служба работает правильно.

`systemctl` позволяет проверить состояние службы, запустить ее, перезапустить и включить автозагрузку.

Для Nginx service-unit обычно называется `nginx`.

Для Apache в Debian/Ubuntu service-unit обычно называется `apache2`.

Проверка состояния web-службы

Для Nginx состояние проверяют так: `systemctl status nginx`.

Для Apache используют: `systemctl status apache2`.

Ожидаемый результат: Active: active (running).

Если видно failed, нужно читать причину в выводе `status` и журналах.

Status показывает состояние службы, но не заменяет проверку порта и клиента.

Запуск и автозагрузка web-службы

Если служба остановлена, ее запускают: `sudo systemctl start nginx`.

Для включения автозагрузки используют: `sudo systemctl enable nginx`.

Можно совместить действия: `sudo systemctl enable --now nginx`.

После enable проверяют автозагрузку: `systemctl is-enabled nginx`.

Для Apache вместо nginx используется apache2.

Restart и reload

restart полностью перезапускает службу и может прервать активные подключения.

Для Nginx пример перезапуска: `sudo systemctl restart nginx`.

reload просит службу перечитать конфигурацию без полного перезапуска, если это поддерживается.

Для Nginx пример мягкого применения: `sudo systemctl reload nginx`.

После restart или reload снова проверяют status и доступ к странице.

Что делать при failed

- Сначала прочитать `systemctl status`, а не повторять `restart` вслепую.
- Если проблема связана с конфигурацией Nginx, полезна проверка: `sudo nginx -t`.
- Для Apache используется проверка конфигурации: `sudo apachectl configtest`.
- После исправления конфигурации применяют `reload` или `restart`.
- Если служба не стартует из-за занятого порта, нужно проверить слушающие порты через `ss`.

7. Проверка доступности web-сервиса

- Доступность web-сервиса проверяется последовательно: служба, порт, firewall, локальный запрос, запрос с клиента.
- Нельзя считать работу подтвержденной только потому, что пакет установлен.
- Нужно доказать, что сервер слушает порт и клиент реально получает страницу.
- Такой порядок формирует профессиональную диагностику вместо случайных попыток.

Проверка порта через ss

`ss` показывает сетевые сокеты, то есть активные сетевые подключения и слушающие порты.

Для проверки HTTP используют: `ss -tulpen | grep ':80'`.

Ключ `-t` показывает TCP, `-u` UDP, `-l` listening, `-p` процесс, `-e` расширенную информацию, `-n` числовой вывод.

Ожидаемый результат: порт 80 слушает процесс `nginx` или `apache2`.

Если порт не слушается, клиентский браузер не сможет получить страницу.

Локальная проверка через curl

curl - инструмент HTTP-запросов из командной строки.

Локальная проверка выполняется на сервере: curl http://127.0.0.1.

Ожидаемый результат: в выводе есть HTML-код или текст опубликованной страницы.

Если локально страница открывается, web-служба работает на сервере.

Если с клиента доступа нет, дальше проверяют IP-адрес, firewall и сеть.

Проверка с клиентской машины

Клиентская проверка подтверждает, что сервис доступен не только самому серверу.

В браузере открывают `http://IPv4-адрес_сервера`.

Например: `http://192.168.10.20`.

Если страница открылась, путь клиент-сеть-сервер-web-служба работает.

Если нет, диагностику начинают с `ping`, `firewall` и проверки порта на сервере.

Firewall и публикация HTTP

Firewall ограничивает сетевой доступ к портам сервера.

UFW - Uncomplicated Firewall, простой интерфейс настройки firewall в Ubuntu/Debian-подобных системах.

Для разрешения HTTP используют: `sudo ufw allow 80/tcp`.

Проверка правил выполняется через: `sudo ufw status verbose`.

Если UFW активен и порт 80 не разрешен, web-служба может работать локально, но быть недоступной клиенту.



ПОДГОТОВКА LINUX-СЕРВЕРА

перед установкой службы

Правильно подготовленный сервер — это стабильность, безопасность и предсказуемость работы любой службы.



Безопасность



Стабильность



Управляемость

ЗАЧЕМ ГОТОВИТЬ СЕРВЕР?

- ✓ Единые стандарты именования и адресации.
- ✓ Актуальные обновления безопасности.
- ✓ Корректные время и DNS — основа работы служб.
- ✓ Минимум лишнего софта — меньше рисков.
- ✓ Чистая среда — проще сопровождение и отладка.

МИНИМАЛЬНЫЕ ТРЕБОВАНИЯ

- Рекомендуемые ресурсы (для учебных целей)
- CPU: 2 vCPU и выше
 - RAM: 2-4 GB
 - Диск: 20 GB+
 - Доступ: консоль/SSH с правами sudo

ПОРЯДОК ПОДГОТОВКИ СЕРВЕРА

1 ПРОВЕРКА ПОДКЛЮЧЕНИЯ

Убедитесь, что сервер доступен по сети.



```
$ ping -c 3 <IP_сервера>
$ ssh user@<IP_сервера>
```

2 УСТАНОВКА ИМЕНИ ХОСТА

Имя сервера должно быть понятным и уникальным.

host01.example.local

```
# временно
$ sudo hostnamectl set-hostname host01

# постоянно
$ sudo vi /etc/hostname
host01.example.local

$ sudo vi /etc/hosts
127.0.0.1 localhost
127.0.1.1 host01.example.local
```

3 НАСТРОЙКА СЕТЕВОГО АДРЕСА

Настройте статический IPv4-адрес, шлюз и DNS.



Пример для netplan (Ubuntu):

```
$ sudo vi /etc/netplan/01-netcfg.yaml

network:
  version: 2
  ethernet:
    ens160:
      dhcp4: no
      addresses: [192.168.10.20/24]
      gateway4: 192.168.10.1
      nameservers:
        addresses: [192.168.10.2, 8.8.8.8]
```

\$ sudo netplan apply

4 ОБНОВЛЕНИЕ ПАКЕТОВ

Обновите индекс репозитория и установленные пакеты.



```
# Debian / Ubuntu
$ sudo apt update
$ sudo apt upgrade -y

# RHEL / CentOS / Rocky
$ sudo dnf update -y
```

5 УСТАНОВКА БАЗОВЫХ УТИЛИТ

Установите утилиты, которые понадобятся для диагностики и администрирования.



```
# Debian / Ubuntu
$ sudo apt install -y \
vim curl wget net-tools \
dnsutils iproute2 \
lsuf htop unzip ufw

# RHEL / CentOS / Rocky
$ sudo dnf install -y \
vim curl wget net-tools \
bind-utils iproute \
lsuf htop ufw
```

6 ПРОВЕРКА КЛЮЧЕВЫХ ПАРАМЕТРОВ

- Имя хоста
hostnamectl
- IP-адрес
ip -4 addr
- Маршрут по умолчанию
ip route
- DNS-серверы
resolvectl status
- Время и синхронизация
timedatectl status
- Открытые порты
ss -tuln

```
$ hostnamectl
Static hostname: host01.example.local
P ip -4 addr show ens160
inet 192.168.10.20/24 brd 192.168.10.255 ...
$ ip route
default via 192.168.10.1 dev ens160
$ resolvectl status
DNS Servers: 192.168.10.2
8.8.8.8
$ timedatectl status
NTP synchronized: yes
$ ss -tuln
State Recv-Q Send-Q Local Address:Port
LISTEN 0 128 0.0.0.0:22
```

7 БАЗОВАЯ БЕЗОПАСНОСТЬ

- Создайте пользователя для администрирования
- Используйте ключи SSH (аутентификация по ключу)
- Отключите вход root по паролю
- Настройте UFW (или firewall)
- Проверьте статус службы SSH

```
# создать пользователя
$ sudo adduser adminuser
$ sudo usermod -aG sudo adminuser

# ключи SSH
$ ssh-keygen -t ed25519
$ ssh-copy-id adminuser@host01

# запрет входа root по паролю
$ sudo vi /etc/ssh/sshd_config
PermitRootLogin no
PasswordAuthentication no
$ sudo systemctl reload sshd

# firewall (пример)
$ sudo ufw allow OpenSSH
$ sudo ufw enable
$ sudo ufw status
```

8 ВРЕМЯ И DNS — ОСНОВА ВСЕГО

- Время
- Корректное время важно для сертификатов, логов и аутентификации.

```
$ timedatectl set-ntp true
$ timedatectl status
```

- DNS
- Проверьте разрешение имен как для внешних, так и для внутренних ресурсов.

```
$ ping -c 2 google.com
$ nslookup example.com
$ getent hosts example.com
```

9 ЧИСТАЯ СРЕДА

- Удалите ненужные пакеты и службы.
- Отключите неиспользуемые службы.
- Освободите место на диске.

```
$ sudo systemctl list-unit-files --state=enabled
$ sudo systemctl disable <service>
$ sudo apt autoremove -y
$ df -h
```

10 ФИНАЛЬНАЯ ПРОВЕРКА

- ✓ Сервер доступен по SSH.
- ✓ Имя хоста задано и разрешается.
- ✓ Статический IP-адрес работает.
- ✓ Шлюз и DNS настроены корректно.
- ✓ Пакеты обновлены.
- ✓ Время синхронизировано.
- ✓ Базовые утилиты установлены.
- ✓ Firewall активен.
- ✓ Журналы не содержат ошибок.

ДОКУМЕНТАЦИЯ СЕРВЕРА

Зафиксируйте параметры в паспорте сервера:

- Имя хоста:
- IP-адрес / маска:
- Шлюз:
- DNS:
- Назначение сервера:
- Ответственный:
- Дата подготовки:

ПОЛЕЗНЫЕ СОВЕТЫ

- Используйте осмысленные имена: web01, dns01 и т.д.
- Всегда работайте под обычным пользователем через sudo.
- Регулярно обновляйте систему безопасности.
- Храните конфигурации под контролем (git, backup).



ВАЖНО ПОМНИТЬ

Подготовка сервера — обязательный шаг перед установкой любой службы. Экономит время, предотвращает ошибки и упрощает сопровождение.

ИТОГОВАЯ ЦЕПОЧКА



Доступность



Имя и сеть



Обновления



Утилиты



Безопасность



Проверки



Документация



Готов к службе

Диагностика недоступной страницы

Если browser показывает timeout, сначала проверить IP-связность до сервера.

Если connection refused, порт может не слушаться или служба остановлена.

Если 403 Forbidden, чаще всего проблема в правах или настройке document root.

Если 404 Not Found, сервер работает, но запрошенный файл или путь не найден.

Если локально работает, а с клиента нет, чаще всего виноваты firewall, адрес или сеть.

8. Журналы web-сервера и документирование

Журналы позволяют доказать, что клиент обращался к серверу, и увидеть ошибки.

Документирование нужно, чтобы другой администратор мог повторить проверку.

Для первой web-службы достаточно зафиксировать имя сервера, IP, пакет, службу, порт, firewall и команды проверки.

Хороший отчет содержит не только команды, но и выводы по результатам.

Access log

Access log фиксирует обращения клиентов к web-серверу.

Для Nginx типичный путь: `/var/log/nginx/access.log`.

Для Apache типичный путь:
`/var/log/apache2/access.log`.

Для просмотра последних строк используют: `sudo tail -n 20 /var/log/nginx/access.log`.

Ожидаемый результат: после запроса клиента появляется строка с IP-адресом клиента и HTTP-статусом.

Error log

Error log фиксирует ошибки web-сервера.

Для Nginx типичный путь: `/var/log/nginx/error.log`.

Для Apache типичный путь:
`/var/log/apache2/error.log`.

Для наблюдения в реальном времени используют:
`sudo tail -f /var/log/nginx/error.log`.

Если страница не открывается из-за прав, пути или конфигурации, подсказка часто появляется именно здесь.

Журнал службы через journalctl

journalctl читает журналы systemd-служб.

Для Nginx используют: journalctl -u nginx -b --no-pager.

Ключ -u выбирает службу, -b ограничивает текущей загрузкой, --no-pager выводит текст сразу.

Для Apache используют: journalctl -u apache2 -b --no-pager.

Этот журнал полезен при ошибках запуска, reload или restart.

Паспорт сервера

Паспорт сервера - краткая техническая карточка, которая описывает установленный сервер.

В него включают `hostname`, IPv4-адрес, роль, установленный пакет, имя службы, порт, `firewall`-правило.

Также фиксируют пути к `web`-каталогу и журналам.

Отдельно записывают команды проверки и итог: страница открывается с клиента или нет.

Паспорт помогает не терять контекст между лабораторными работами.

Минимальный чек-лист результата

- hostname задан и зафиксирован.
- IPv4-адрес сервера известен и доступен клиенту.
- Web-пакет установлен, служба active (running).
- TCP-порт 80 слушается нужным процессом.
- Firewall разрешает HTTP, страница открывается с клиента.
- Access/error logs проверены, результат внесен в отчет.

Итоги лекции

- Linux-сервер предприятия рассматривается как управляемая роль в инфраструктуре.
- Сервисная модель связывает пакет, конфигурацию, службу, порт и журнал.
- Публикация web-сервиса требует подготовки сервера, установки пакета, запуска службы и проверки доступа.
- Проверка должна идти последовательно: status, порт, firewall, curl, браузер, журналы.
- Документирование превращает выполненную настройку в воспроизводимый результат.

Контрольные вопросы

1. Какую роль выполняет Linux-сервер в инфраструктуре предприятия?
2. Какие серверные роли может выполнять Linux в учебной корпоративной сети?
3. Как связаны пакет, конфигурационный файл, служба, порт и журнал?
4. Почему серверу нужно задавать понятный hostname и статический IPv4-адрес?
5. Как проверить, что web-служба запущена и слушает нужный порт?
6. Для чего используется firewall при публикации web-сервиса?
7. Какие сведения следует включить в паспорт Linux-сервера?